

Generalized Deduplication: Lossless Compression by Clustering Similar Data

Prasad Talasila, Daniel E. Lucani

Department of Engineering, DIGIT, Aarhus University, Aarhus, Denmark
{ prasad.talasila, daniel.lucani }@eng.au.dk

Abstract—This paper proposes generalized deduplication, a concept where similar data is systematically deduplicated by first transforming chunks of each file into two parts: a basis and a deviation. This increases the potential for compression as more chunks can have a common basis that can be deduplicated by the system. The deviation is kept small and stored together with an identifier to its chunk, e.g., hash of a chunk, in order to recover the original data without errors or distortions. This paper characterizes the performance of generalized deduplication using Golomb-Rice codes as a suitable data transform function to discover similarities across all files stored in the system. Considering different synthetic data distributions, we show in theory and simulations that generalized deduplication can result in compression factors of 300 (high compression), i.e., 300 times less storage space, and that this compression is achieved with 60,000 times fewer data chunks inserted into the system compared to classic deduplication (compression gains start earlier). Finally, we show that the table/registry to recognize similar chunks is 10,000 times smaller for generalized deduplication compared to the table in classic deduplication techniques, which will result in less RAM usage in the storage system.

Index Terms—generalized deduplication, golomb rice, geometric distribution, data deduplication

I. INTRODUCTION

Internet of Things (IoT) devices, virtual and augmented reality, and video applications are expected to generate exponentially growing volumes of data that need to be stored in Cloud systems. Dictionary-based lossless compression schemes [1]–[3], and combinations with Huffman codes [4], are popular for text and multimedia compression. Golomb-Rice (GR) codes [5]–[7] and its' variations, e.g., [8], are widely used in audio and video codecs, e.g., MPEG [9]. However, these compression algorithms are not suitable for compression at a file system or Cloud system level due in large part to their inability to exploit redundancies across multiple files without introducing dependencies between those files at the time of accessing or modifying them.

To compensate for these limitations, existing approaches rely on data deduplication to find an exact matching of data chunks from the same or even different files stored in a file system or Cloud system [10]–[13]. These *classic deduplication* schemes typically produce relatively large deduplication tables that help map the file's with their corresponding data chunks as well as the location of the chunks and their ID for easy recognition and comparison (e.g., a hash of the chunk). An example is ZFS which potentially produces a 5GB deduplication table for every TB of data stored in the system [13]. Rabin fingerprints have been considered to identify similarities in entire files [14] or file chunks [15] in order to perform a Delta encoding (difference) of two files or chunks of data in order to compress them. This allows to deduplicate the difference between chunks that are similar

rather than the chunks themselves, but creates a dependency across several data chunks in the system, which can affect the access times and effect of changes to the content. Identifying and exploiting similarities in the chunks without creating dependencies on previous chunks available in the system, i.e., each chunk can be accessed/modified indepently from others, is one of the problems addressed in this paper.

In this paper, we propose a technique named *generalized deduplication* that transforms data chunks from each file into two parts, namely, a basis and a deviation from the basis. The goal is to exploit similarities in the data, namely, chunks with the same basis, in order to have more opportunities to match chunks and result in higher compression potential. Our approach deduplicates the basis, which is the common part of many chunks with different original values. The deviation is small in size and contains the missing information that needs to be used to result in a proper recovery of the original chunk.

The main contributions of this paper are:

- 1) a mathematical formulation of compression gain possible with generalized deduplication and analysis using synthetic distributions for the data;
- 2) the use of Golomb Rice (GR) codes as a valid transformation function for generalized deduplication; and
- 3) a higher compression gain with the same number of data chunks and smaller deduplication tables for generalized deduplication vis-à-vis classic deduplication.

The rest of this paper is organized as follows. Section II describes the system model for generalized deduplication and its core building blocks. Classic deduplication is shown to be a special case of generalized deduplication. Section II provides a mathematical analysis of generalized deduplication under specific data input distributions. Section IV shows numerical results and simulations showing the concrete gains of generalized deduplication, while Section V focuses on conclusions and directions for future work.

II. SYSTEM MODEL

We consider a system receiving f files. We assume that each file is a concatenation of n data chunks, i.e., $c_1 c_2 \dots c_n$, where each data chunk (in short, *chunk*) c_i has a length of m bits. The total number of chunks (N) processed by the system is $N = fn$. We also consider that each file has some additional metadata associated to it, e.g., owner, file permissions, which requires an additional s_{md} bits for storage. Let us define C as the set of all unique chunks, where $|C| = 2^m$.

We consider that a system implementing generalized deduplication will support two key operations: *store* and *load*. Figure 1 provides a graphical illustration of a file system using generalized deduplication. When receiving a store request, the

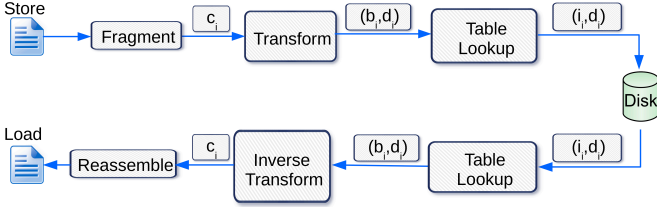


Fig. 1. Graphical representation of the generalized deduplication technique.

system fragments the incoming file into chunks. Each chunk c_i will be transformed into a (b_i, d_i) pair using a bijective transform function, where b_i is a basis and d_i is a deviation from the basis. The sizes in bits of a basis and deviation are s_b and s_d , respectively.

The system maintains a table to index all unique basis elements (called *index table* or *dedup table*). For each (b, d) pair corresponding to chunk c , the system will identify if the content of b is already part of the table (e.g., using a hash function). If the value of b is already in the table, then the corresponding index $\mathcal{I}$ of size $s_{\mathcal{I}-g} (< s_b)$ bits for that basis b is returned. Otherwise, the basis is included and a new index ($\mathcal{I}$) is generated and returned. The table is empty when the system starts, i.e., when no chunks have been processed. At this point, a chunk c_i will be represented in the system as a pair $(\mathcal{I}_i, d_i)$ and an entire file will be represented as a concatenation of such pairs. Since $s_{\mathcal{I}-g} + s_d \ll m$ bits, the system has a significant compression potential. This is not only due to the reduction per file, but by the fact that the system uses the table of indexes for all files stored in the system. This means that compression occurs across all information stored in the system. Finally, $s_{\mathcal{I}-g}$ denotes the number of entries (i.e., size) of the table after loading f files totalling N chunks.

Upon a load file request, the system will access the file representation in terms of $(\mathcal{I}, d)$ pairs, search for the actual basis b associated to each index $\mathcal{I}$ and then apply the inverse transform of each (b, d) pair into the corresponding chunk. Finally, the system will concatenate the n chunks in their original order to reconstruct the file and retrieve the necessary metadata. The search for the basis is carried out as a table look up for each index $\mathcal{I}$.

A. Transform Function Characteristics

An appropriate selection of the transform function is key to reaping the benefits of generalized deduplication. Any suitable transform function should make the number of active bases, i.e., bases observed in the N chunks, significantly smaller than the number of chunks. To avoid data losses, we require that the transform from a chunk c to a pair (b, d) is a bijection. To achieve this, let us define $B = \{b_1, b_2, \dots, b_{|B|}\}$ and $D = \{d_1, d_2, \dots, d_{|D|}\}$ as the set of all unique bases and unique deviations, respectively. That is, $b_i \neq b_j, \forall b_i, b_j \in B, i \neq j$. The transform function organizes all elements of C into disjoint subsets $C_i \subseteq C$, where $\cup_i C_i = C$ and $C_i \cap C_j = \emptyset, \forall i \neq j$. Each disjoint subset C_i of C is mapped to one element $b_i \in B$. No element of B is used to map more than one subset of C and all elements of B are used in the mapping.

Thus, we expect $s_b + s_d = m$. A good transform function will map multiple elements to one basis, i.e., $|C_i| > 1, \forall i$, thus, creating the potential to compress data in the system.

In this paper, we explore the use of Golomb-Rice (GR) codes [5], [6] as a suitable transform function. First, note that GR codes are prefix codes which can separate a chunk into two parts and that multiple chunks are mapped to each basis. In our context, the transform function for each chunk c_i is given by $c_i = b_i \times 2^k + d_i$, where k is the parameter of the GR codes. Note that $|C_i| = 2^k, \forall C_i \in C$, i.e., the number of elements mapped to each basis is the same.

B. Computational Complexity

In order for the generalized deduplication to compare favorably with classic deduplication, the transform and the inverse transform functions used by generalized deduplication must be easy to compute. Since simple GR codes only require a separation of the chunk into its most-significant bytes, which are assigned to the basis, and the least-significant bytes, which are assigned to the deviation. This means that we do not require complex transformations, only a splitting of the original data into two parts.

C. Classic Deduplication as a Special Case

In classic deduplication [11], [12], each file is also represented as a sequence of chunks. Chunks are deduplicated with the help of a table linking an index $\mathcal{I}$ to each unique chunk c . Files are represented as a concatenation of indexes or keys in the table. Thus, in deduplication there is no splitting into basis and deviation, i.e., no transform function is applied. The equivalent transform function from the perspective of generalized deduplication would be a one-to-one mapping, where $b = c$ with a constant value for the deviation ($d = 0$) which does not provide any valuable information, i.e., can be discarded. Thus, it constitutes a special case of our generalized framework. For our analysis, we define $s_{\mathcal{I}-c}$, and s_{t-c} to be the size in bits of the index and table, respectively, for the specific case of classic deduplication.

III. ANALYSIS OF GENERALIZED DEDUPLICATION

A. Performance Metrics

The data compression achieved by a deduplication technique can be quantified using a metric called *compression gain* or *compression factor*, which is defined as the ratio of storage space required for the uncompressed data to the storage required for the output of generalized deduplication (or classic deduplication). We define G_g and G_c as the compression gains achieved by the generalized and the classic deduplication techniques, respectively.

Note that each uncompressed file occupies $(nm + s_{md})$ bits on the disk, while all the files require $f(nm + s_{md})$ bits. Using generalized deduplication, each file originally containing n chunks now occupies $n(s_{\mathcal{I}-g} + s_d) + s_{md}$ bits. Generalized deduplication also uses a table which requires $s_{\mathcal{I}-g}(s_{\mathcal{I}-g} + s_b)$ bits. Thus, the total storage required is $f(n(s_{\mathcal{I}-g} + s_d) + s_{md}) + s_{\mathcal{I}-g}(s_{\mathcal{I}-g} + s_b)$ bits. Therefore, the compression gain G_g can be formulated as

$$G_g = \frac{m + \frac{s_{md}}{n}}{s_{\mathcal{I}-g} + s_d + \frac{s_{md}}{n} + \frac{s_{\mathcal{I}-g}}{N}(s_{\mathcal{I}-g} + s_b)}. \quad (1)$$

An upper bound for G_g can be calculated as $G_g \leq G_g|_{f,n \rightarrow \infty} = \frac{m}{s_{I-g} + s_d}$. For the special case of classic deduplication, the compression gain is

$$G_c = \frac{m + \frac{s_{md}}{n}}{s_{I-c} + \frac{s_{md}}{n} + \frac{s_{I-c}}{N}(s_{I-c} + m)}. \quad (2)$$

Theorems 2 and 1 show that our scheme is strictly better in both table size and compression gains.

Theorem 1: Generalized deduplication has better or equal compression gain than classic deduplication for the same f files with n chunks each, i.e., $G_g \geq G_c$.

Proof: Proven by contradiction assuming $G_g < G_c$, and using the fact that s_{I-g} , N , s_d , δ , s_d , s_{I-c} , and s_b are positive. ■

Theorem 2: The table size in generalized deduplication is smaller or equal to that in classic deduplication for the same f files with n data chunks each, i.e., $s_{I-g} \leq s_{I-c}$.

B. Data Model

In this paper, we assume that the content or value of a chunk is independent and identically distributed from chunks in the same file and across files and assume a geometric distribution. The probability mass function (PMF) of the geometric distribution for the input chunks is

$$P(X = h) = (1 - p)^h p, h = 0, 1, \dots \quad (3)$$

where h denotes the decimal value related to the content of the chunk of size m bits without loss of generality. This means that $h = 2$ is a string of m bits of the form 00...010. The parameter p indicates the probability of selecting $h = 0$, i.e., the bit string 0...0.

Classical Deduplication: Given a sufficiently large N we will eventually need an index of size m -bits for uniquely representing all possible chunks. However, certain chunks occur rarely given our data model. We define *marginal chunks* as the set of chunks which jointly have a probability of occurrence of less than ε . Let l represent the first l -elements of the geometric distribution such that the probability of one of those elements being selected is $(1 - \varepsilon)$. Let s_I represent the size in bits of the index used in the table. Then,

$$\sum_{h=l}^{2^m} P(X = h) < \varepsilon. \quad (4)$$

Substituting Eq. (3) in Eq. (4) and solving for l , we get

$$l > l_{\min} = \log_{(1-p)} \varepsilon, \quad (5)$$

which provides a closed form expression for the threshold of non-marginal inputs. Note that chunks with value $h \geq l_{\min}$ have a low probability of appearing in the f files. $l_{\min}$ can be used to provides an estimate of the total number of active values for chunks as well as the minimum required length of index ($s_I^{\min}$) for indexing only the non-marginal inputs:

$$s_I^{\min} = \log_2 l_{\min} = \log_2 \log_{(1-p)} \varepsilon. \quad (6)$$

The above discussion for $s_I^{\min}$ provides a direct indication of the index size in bits, where $l_{\min}$ indicates the minimum size of the table for classic deduplication.

Generalized deduplication: In this case, only bases count towards the size of the table and, thus, for the index size. For the first $l_{\min}$ non-marginal chunks, given by Eq. (5), the number of bases generated by the GR code is

$$l_g = \lceil l_{\min} / 2^k \rceil. \quad (7)$$

This comes from the fact that GR codes will cluster the first 2^k chunk values ($h = 0, 1, 2, \dots, 2^k - 1$) and map them to the same basis (with value 0), the second 2^k chunk values ($h = 2^k, 2^k + 1, \dots, 2 \cdot 2^k - 1$) to another basis (with value 1), and so on. Thus, there are l_g non-marginal bases using GR codes. Note that the size of k reduces the number of elements, l_g , in the table exponentially. Even a moderate k value in GR will yield a significant reduction in the size of the deduplication table with respect to classic deduplication. For example, $k = 8$ bits makes l_g 256 times smaller than $l_{\min}$.

We can express the minimum required length of index ($s_{I-g}^{\min}$) for generalized deduplication as

$$s_{I-g}^{\min} = \log_2 \frac{l_{\min}}{2^k} = s_I^{\min} - k, \quad (8)$$

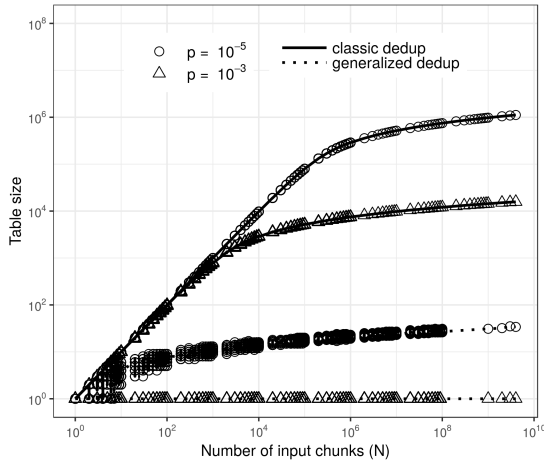
which means that the size of k has a direct impact on the number of bits required for representing the active bases.

IV. NUMERICAL ANALYSIS

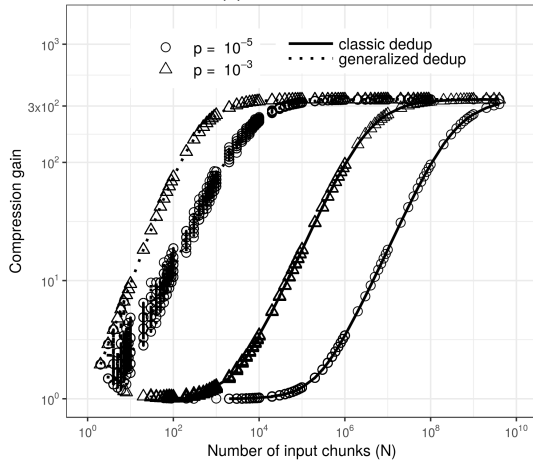
For the numerical results, we use a single file ($f = 1$) input with a chunk size $m = 8192$ bits and $s_{md} = 320$ bits of file metadata. For generalized deduplication, we use a GR code with $k = 16$ bits, which results in a deviation size $s_d = 16$ bits. We generate fifty different input data chunk streams for a given deduplication configuration, i.e., each data point considers $50 \cdot N$ chunks in total. Both the marginal and non-marginal input chunks are subjected to deduplication. In our Figures, *dedup* is used as a shorthand for deduplication. We consider the performance of classic and generalized deduplication for different values of N and data characteristics given by p .

Figure 2a shows that the size of the table to maintain the mapping is significantly smaller in generalized deduplication than in classic deduplication. For $N > 10^6$, the table of generalized deduplication is more than 10,000 times smaller than table of the classic deduplication, i.e., $s_{I-c} > 10000 s_{I-g}$. Using generalized instead of classical deduplication would result in much smaller table sizes and fewer RAM restrictions in real systems like ZFS [13]. Figure 2a also shows that classic deduplication has a maximum table size of $s_{I-c} \approx 10^6$, thus requiring 24 bits for index representation, while generalized deduplication has a maximum table size of $s_{I-g} \approx 10^2$, requiring only 8 bits for basis index representation.

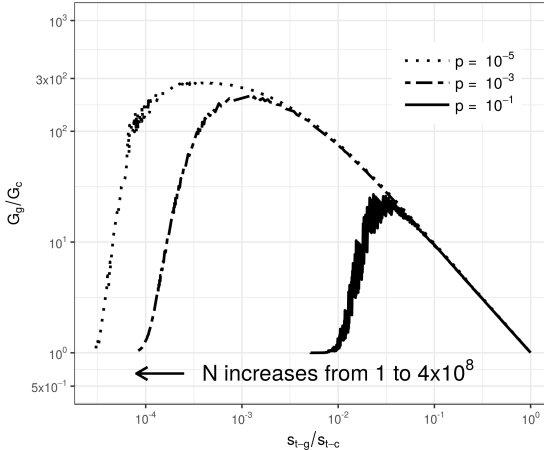
Figure 2b shows the gains in compression of the generalized deduplication vis-à-vis classic deduplication. For $p = 10^{-5}$, $G_g > G_c$ for all $N < 10^{10}$. It is also important to emphasize that for the same p classic deduplication requires 60,000 times more chunks to be inserted in the system to reach the same gain as generalized deduplication. Moreover, as p decreases (i.e., there are more non-marginal elements) the N at which classic deduplication starts to see gains (i.e., $G_c > 1$) grows significantly, while for the generalized case the necessary N is much lower and grows slowly as p decreases.



(a) Table size



(b) Compression gain.



(c) Relative Compression gain vs. relative table size

Fig. 2. Performance of classic and generalized deduplication

Figure 2c shows the trade-off between performance gain and table size when comparing the the ratio of compression gains and table sizes for generalized and classic deduplication. For $N = 1$, $G_d/G_c = 1$ and $s_{t-g}/s_{t-c} = 1$, which is consistent with the fact that only one chunk in the system has no compression potential under the two strategies considered (no matches are possible yet). As N increases, s_{t-g}/s_{t-c} decreases in value while G_d/G_c first increases and, ultimately,

decreases until $G_d/G_c = 1$ for sufficiently large N . For $N \approx 10^6$ and $p = 10^{-3}$ (10^{-5}) the compression gain provided by generalized deduplication is 200 (300) times better than the compression gain provided by classic deduplication.

V. CONCLUSIONS

In this paper, we proposed generalized deduplication a key alternative for lossless compression of data at a Cloud scale. The key idea behind generalized deduplication is the transformation of the incoming data into a series of chunks represented as basis and deviation pairs prior to subjecting the bases to indexing and deduplication. Our analysis shows that the use of generalized deduplication with GR codes as the transform function results in the reduction of deduplication table size by a factor of up to 10,000. Generalized deduplication also was shown to have the potential to compress the data up to 300 times more efficiently than classic deduplication and starting to have compression potential for a small to moderate number of data chunks introduced in the system. Similarities are characterized systematically and without the need for comparing against or calculating differences to other blocks to achieve gains as is done in previous work [14], [15].

ACKNOWLEDGEMENTS

This work was partially financed by the SCALE-IoT Project (Grant No. 7026-00042B) granted by the Independent Research Fund Denmark, by the Aarhus Universitets Forskningsfond (AUFF) Starting Grant Project AUFF-2017-FLS-7-1, and Aarhus Universitys DIGIT Centre.

REFERENCES

- [1] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Trans. on Info. Theory*, vol. 24, no. 5, pp. 530–536, 1978.
- [2] T. A. Welch, "Technique for high-performance data compression," *Computer*, no. 52, 1984.
- [3] J. A. Storer and T. G. Szymanski, "Data compression via textural substitution," *Journal of the ACM*, vol. 29, no. 4, pp. 928–951, 1982.
- [4] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [5] S. Golomb, "Run-Length Encodings (Corresp.)," *IEEE Trans. on Info. Theory*, vol. 12, no. 3, pp. 399–401, 1966.
- [6] R. Rice and J. Plaunt, "Adaptive Variable-Length Coding for Efficient Compression of Spacecraft Television Data," *IEEE Trans. on Comm. Tech.*, vol. 19, no. 6, pp. 889–897, 1971.
- [7] R. Gallager and D. Van Voorhis, "Optimal Source Codes for Geometrically Distributed Integer Alphabets (corresp.)," *IEEE Trans. on Info. Theory*, vol. 21, no. 2, pp. 228–230, 1975.
- [8] J. Teuhola, "A compression method for clustered bit-vectors," *Info. Proc. Letters*, vol. 7, no. 6, pp. 308–311, 1978.
- [9] T. Liebchen and Y. A. Reznik, "Mpeg-4 als: An emerging standard for lossless audio coding," in *Data Comp. Conf. (DCC)*, 2004, pp. 439–448.
- [10] W. Xia, H. Jiang, D. Feng, F. Dougliis, P. Shilane, Y. Hua, M. Fu, Y. Zhang, and Y. Zhou, "A Comprehensive Study of the Past, Present, and Future of Data Deduplication," *Proc. of the IEEE*, vol. 104, no. 9, pp. 1681–1710, 2016.
- [11] A. T. Clements, I. Ahmad, M. Vilayannur, J. Li *et al.*, "Decentralized Deduplication in SAN Cluster File Systems," in *USENIX A. Tech. Conf.*, 2009, pp. 101–114.
- [12] S. Quinlan and S. Dorward, "Venti: A New Approach to Archival Storage," in *FAST*, vol. 2, 2002, pp. 89–101.
- [13] C. Gonzalez. (2013) ZFS: To Dedupe or not to Dedupe. [Online]. Available: <https://constantin.glez.de/2011/07/27/zfs-to-dedupe-or-not-dedupe>
- [14] F. Dougliis and A. Iyengar, "Application-specific delta-encoding via resemblance detection," in *USENIX A. Tech. Conf.*, 2003, pp. 113–126.
- [15] P. Kulkarni, F. Dougliis, J. LaVoie, and J. M. Tracey, "Redundancy elimination within large collections of files," in *USENIX A. Tech. Conf.*